Reading Pict.java, Again

Name: _____ Date: _____

Some of `Pict.java`, exerpted verbatim from the source file.

```
final class Px {
  private Px() {}
  static int of(double v) {
    if (Double.isNaN(v)
        || Double.isInfinite(v)) {
      throw new IllegalArgumentException(
        "measurement must be a number, got "
        + v);
    }
    return (int) Math.round(v);
  }
  static int size(String what, int v) {
    if (v < 0) {
      throw new IllegalArgumentException(
        what + " is negative: " + v);
    }
    return v;
  }
}
```

```
record Circle(int radius, Color color)
    implements Pict {
  Circle { Px.size("radius", radius); }
  Circle(int radius, String color) {
    this(radius, Colors.named(color));
  }
  Circle(double radius, String color) {
    this(Px.of(radius), Colors.named(color));
  }
  // ...
}
// Rect has the same three constructors,
// with a width and a height.
```

Which constructor?

Q1. For each line, write the parameter types of the constructor Java runs *first*, and what prints.

you wrote	runs first	prints
`new Circle(10, "red")`		
`new Circle(10.5, "red")`		
`new Rect(3, 4.5, "red")`		

A crash, three calls deep

Running `Pict p = new Circle(-0.6, "red");` in `main` crashes with:

```
Exception in thread "main" java.lang.IllegalArgumentException: radius is negative: -1
  at Px.size(Pict.java:90)
  at Circle.<init>(Pict.java:143)
  at Circle.<init>(Pict.java:145)
  at Main.main(Main.java:3)
```

(`<init>` is how a stack trace names a constructor.)

Q2. Draw the stack at the moment `Px.size` throws. Write each call as it appears in the code, and the values of its parameters.

NAMES (THE STACK) HEAP

_____

what = _____ v = _____

_____

radius = _____ color = _____

_____

radius = _____ color = _____

main()
p = *not set yet*

top level

Q3. The radius written was `-0.6`, but the message says `-1`. Which line turned one into the other?

What Px.of gives back

Q4. What does each call return? If it throws, write the message.

`Px.of(2.5)`	
`Px.of(-2.5)`	
`Px.of(2.49)`	
`Px.of(Double.NaN)`	
`Px.of(1e10)`	

Q5. `(int) 1e10` would be `2147483647`, the biggest `int`. But `Px.of(1e10)` is `1410065408`. What's different about the way `Px.of` converts?

Who gets called on what

Q6. Is each one called on an object, or on a class? Circle one.

`Px.of(2.5)` object / class

`pic.width()` object / class

`Math.round(2.5)` object / class

`Colors.named("red")` object / class

`r.colorAt(0, 0)` object / class

`"trash".length()` object / class

Raster

```
class Raster {
  private final Color[][] pixels;
  private final int width, height;
  // ... constructors that check and copy the pixels
  int width() { return this.width; }
  Color colorAt(int x, int y) { return this.pixels[y][x]; }
  public boolean equals(Object other) {
    // ... same size, and every row Arrays.equals the other's
  }
}
```

Q7. In your own program, `r` is a `Raster`. Which lines compile? Circle one.

`int w = r.width();` compiles / doesn't

`int w = r.width;` compiles / doesn't

`Color c = r.colorAt(0, 0);` compiles / doesn't

`Color[][] all = r.pixels;` compiles / doesn't

Q8. `r1` and `r2` are two `Raster`s made from the same colors. What are `r1.equals(r2)` and `r1 == r2`?

`r1.equals(r2)` _____ `r1 == r2` _____

Q9. Suppose `Raster` were a record, `record Raster(Color[][] pixels, int width, int height)`. Give two things that could go wrong.